



BAHRIA UNIVERSITY KARACHI CAMPUS

Department of Software Engineering

COURSE: SEN-210 **Intr. to Software Engineering** **PROJECT REPORT**

CLASS: BSE – 2B (SPRING - 2025)

BUSPs(Bahria University Society Polls)

Group Members

S. No.	Name	Enrollment #
01	Aisha Noor	02-131242-063
02	Iffat Batool	02-131242-126
03	Kanwal Bibi	02-131242-065

Submitted to:

Course Instructor: Engr. Bushra Fazal

Date: _____01-06-25_____

1. Requirements Analysis

1.1 Functional Requirements

- Secure login system for students and admins.
- Admin functionalities:
 - Schedule, modify, and remove elections.
 - Manage societies, candidates, and voters.
- Voting system:
 - One vote per student per election.
 - Votes are securely submitted.
- Result processing and display:
 - Real-time automated result calculation.

1.2 Non-Functional Requirements

- **Usability:** Simple UI using Java Swing.
- **Performance:** Fast vote count and real-time updates.
- **Maintainability:** Clear documentation and GitHub version control.
- **Scalability:** Can handle data for multiple societies.

2. System Architecture Design

2.1 High-Level Components

- **Frontend:** Java Swing GUI.
- **Backend:** Java logic, database connectivity.
- **Database:** MariaDB for storing users, votes, elections, societies, and related data.
- **Server:** Apache NetBeans IDE for development.

2.2 Subsystem Interfaces

- Login module → Menu
- Menu → Elections, Candidates, Voters
- Voting → Dashboard

3. Software Development Methodology

3.1 Chosen Methodology: Agile

The system is developed in iterative sprints, allowing continuous feedback and improvements.

3.2 Development Process (Sprints)

- **Sprint 1:** Planning and UI design.
- **Sprint 2:** Backend logic and database schema.
- **Sprint 3:** Voting module and admin features.
- **Sprint 4:** Final testing, documentation, and deployment.

3.3 Version Control Strategy

- **Git** is used for version control, allowing the team to track code changes on individual machines.
- **GitHub** is used as the remote repository for collaboration, sharing code, managing pull requests, and handling merge conflicts.

4. Risk Management

4.1 Identified Risks

- Data loss or corruption.
- System crash during voting.
- Unauthorized access or security breach.
- Delay in development due to bugs.

4.2 Risk Exposure (RE)

- **RE = Probability × Impact**
- Example: Data loss (0.3 × High) = Medium-High

4.3 Risk Leverage (RL)

- **RL = (RE(before) - RE(after)) / Cost of mitigation**
- Justifies the benefit of implementing security measures.

4.4 Mitigation Strategies

- Regular database backups.
- Input validation and authentication.
- Exception handling and testing.
- Agile process to detect bugs early.

5. Testing and Validation

5.1 Testing Strategy

- Unit testing for each module (e.g., login, vote casting).
- Integration testing between components.
- System testing with real users.

5.2 Test Environment

- Java Swing GUI on Apache NetBeans IDE.
- MariaDB running on localhost.

5.3 Sample Test Cases

Test Case	Input	Expected Output
Login	Valid ID/Password	Menu (For Admins)
Vote	Valid voter ID and selection	Voting form (For Voter) loaded
Result	After all votes	Accurate result display

6. Documentation and Maintenance

6.1 Documentation Plan

- User manual for students and admins.
- Technical documentation for developers.
- UML diagrams and architecture design.

6.2 Lifecycle Maintenance

- Bug fixes and updates via GitHub.
- Feedback collection and improvements post-deployment.

7. Work Breakdown Structure (WBS)

7.1 Project Planning

Initial roadmap, timeline, and resource allocation.

7.2 Requirement Gathering

Functional and non-functional requirements analysis.

7.3 UI Design

Creating mockups and finalizing the Java Swing GUI.

7.4 Backend Development

Developing logic for elections, voting, login, and admin tools.

7.5 Database Setup

Creating schemas and integrating MariaDB.

7.6 Testing

Performing all phases of software testing.

7.7 Documentation

User guide, developer guide, and project report.

7.8 Final Review and Submission

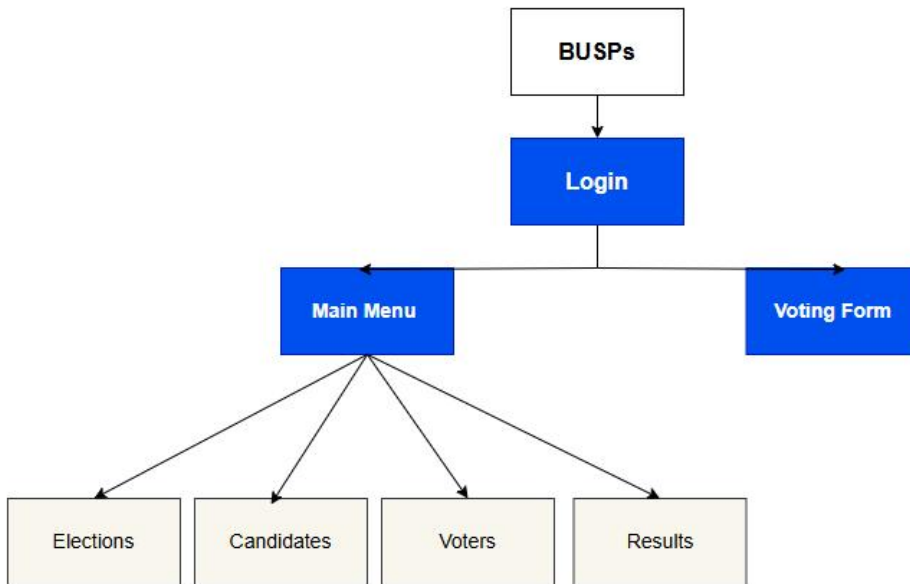
Final testing, debugging, and deployment for demo.



8. UML Diagrams and System Models

8.1 Content Diagram

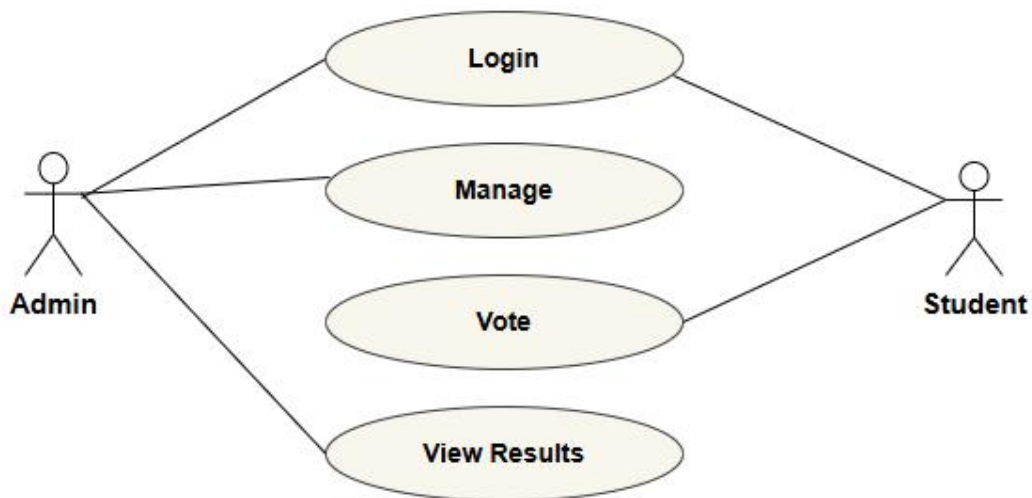
Illustrates project structure: modules like Login, Voting, Admin Panel, Database.



8.2 Use Case Diagram

Actors: Admin, Student

Use Cases: Login, Manage Election, Vote, View Results.

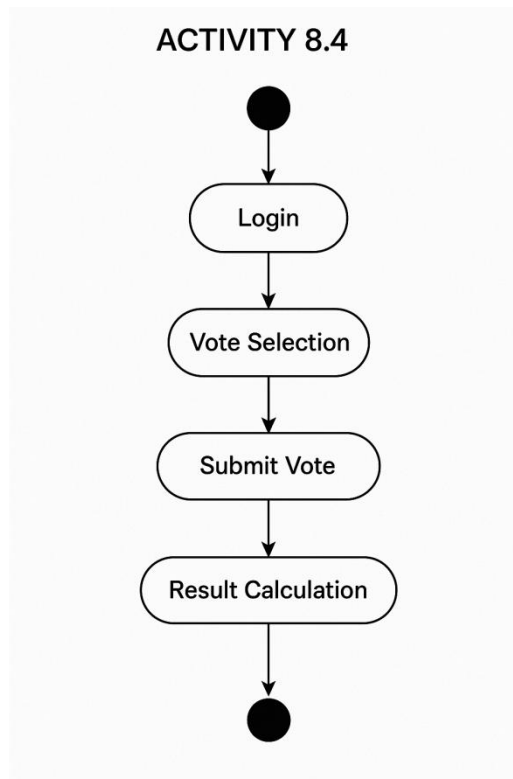


8.3 Class Diagram

Classes: BUSPs, Splash, Login, MainMenu, Dashboard, Elections, Candidates, Voters, Votings
Relationships: Inheritance and Association.

8.4 Activity Diagram (Voting Process)

Login → Vote Selection → Submit Vote → Result Calculation → Display Result



8.5 Sequence Diagram (Creating Poll)

Admin logs in → Navigates to Election module → Creates poll → Poll saved to DB → Confirmation sent.

9. Project Cost Estimation Using COCOMO

9.1 Model Type:

We use the **Basic COCOMO Model** as our project is small-scale and follows a straightforward waterfall-like structure within Agile sprints.

9.2 Mode of Development:

Organic (for small teams with good experience and flexible requirements)

9.3 Effort Estimation (Person-Months):

Formula:

$$\text{Effort (E)} = a \times (\text{KLOC})^b$$

Where:

- $a = 2.4$ (for organic mode)
- $b = 1.05$ (for organic mode)
- KLOC = estimated size in Kilo Lines of Code
- Estimated size: 2,000 lines of code = **2 KLOC**

Calculation:

$$E = 2.4 \times (2)^{1.05} \approx \mathbf{4.92 \text{ Person-Months}}$$

9.4 Time Estimation (Development Time):

Formula:

$$\text{TDEV} = c \times (E)^d$$

Where:

- $c = 2.5$ (for organic mode)
- $d = 0.38$ (for organic mode)
- $E = 4.92$

Calculation:

$$\text{TDEV} = 2.5 \times (4.92)^{0.38} \approx \mathbf{5.89 \text{ Months}}$$

9.5 Team Size Estimation:

$$\text{Team Size} = \text{Effort} / \text{Time} = 4.92 / 5.89 \approx \mathbf{0.83 \approx 1 \text{ person}}$$

Since we worked in a team of **3 students**, and the total development time was **1 month**, this is reasonable, and the extra manpower reduced actual time due to parallel work.

Appendix

Project Scope

Included:

- Election automation.
- Secure voting.
- Admin dashboard.
- Real-time results.

Excluded:

- Integration with external systems.
- Payment handling for society memberships.

Project Abstract

BUSPs is a Java-based voting system for student societies at Bahria University. It ensures efficient, secure, and transparent elections, replacing the manual process.

Development Time

4 Weeks / 28–30 Days

Technologies Used

- **Language:** Java
- **GUI:** Java Swing
- **Database:** MariaDB
- **IDE:** Apache NetBeans 24
- **Version Control:** GitHub, Git